PROBLEMAS E ALGORITMOS
UM ESTUDO SOBRE METAHEURÍSTICA

ANT COLONY OPTIMIZATION

THÁLYS LEMOS CORREA
VITOR DA SILVA FERREIRA

ROTEIRO DA APRESENTAÇÃO

- Fundamentos Teóricos
  - História, Inspiração Biológica e Funcionamento do ACO
- O Problema de Otimização
  - Definição (Touros x Vacas), Grafo Bipartido e Restrições
- Metodologia e Implementação
  - Estratégias em Python, Tratamento de Dados e Algoritmo ACS
- Resultados Experimentais
  - Comparativo de Cenários (12, 18 e 20 usos), Análise de Ganho e Gráficos

HISTÓRIA

- Proposta por Marco Dorigo em 1992 em sua tese de doutorado, um pesquisador italiano pioneiro em inteligência artificial e sistemas inspirados na natureza;
- A Ant Colony Optimization é inspirada em comportamento de formigas reais;
- Utilizada para resolver problemas de otimização combinatória (melhor solução para um conjunto finito);



PRINCÍPIOS E FUNCIONAMENTO

- Usa feromônios como comunicação indireta, permitindo reforço de caminhos eficientes;
- Caminhos ruins perdem atratividade pela evaporação do feromônio;
- A meta-heurística replica esse comportamento: agentes simples constroem soluções cooperativamente;
- As formigas escolhem estados via probabilidade baseada em feromônio + heurística local; Cada formiga gera uma solução completa, onde boas soluções são reforçadas e as ruins são evaporadas para evitar estagnação e mantém exploração do espaço de busca;
- Extensões incluem busca local e técnicas de refinamento para melhorar convergência.

PROBLEMA

- A ACO possui várias variantes, e neste trabalho utilizamos o Ant Colony System (ACS);
- O ACS é uma versão aprimorada do Ant System, proposta por Dorigo e Gambardella em 1997;
- Diferente da ACO clássica, o ACS aplica atualização local do feromônio a cada passo;
- A atualização local aumenta a diversificação da busca durante a construção das soluções;
- O ACS realiza atualização global apenas nas melhores soluções ao final de cada iteração;
- Ele utiliza uma regra de decisão que equilibra exploração e intensificação de caminhos promissores.

DEFINIÇÕES DO PROBLEMA

M = conjunto de touros F = conjunto de vacas

(i, j) = descendente dos dois animais onde $i \in M$ e $j \in F$

$$x_{(i,j)} = \begin{cases} 1 \text{ se a fêmea j cruzar com o macho i} \\ 0 \text{ caso contrário} \end{cases}$$

Para as restrições de acasalamento, temos que para cada $j \in F : \sum_{i \in M_j} x_{(i,j)} = 1$, onde $M \subseteq M$ é o subconjunto de machos que podem cruzar com a fêmea j

$c_{(x,y)(w,v)}$ = coeficiente de coancestralidade (entre 0 e 1)

FUNÇÃO OBJETIVO

$$P_{(x)} = \min \sum_{(x,y)} \sum_{(w,v)} c_{(x,y)(w,v)} x_{(x,y)} x_{(w,v)}$$

DEFINIÇÕES EM ACS

- Cada formiga artificial percorre um grafo que conecta o conjunto de fêmeas F ao conjunto de machos M;
- Cada aresta (i, j) representa a decisão de acasalamento produzindo o descendente $x_{(i,j)}$;
- O nível de feromônio associado a cada aresta, denotado por $\tau_{(i,j)}$, indica o grau de atratividade daquela combinação de acasalamento com base nas iterações anteriores;
- Heurística de visibilidade $\eta_{(i,j)}$ é definida a partir dos índices genéticos dos animais, de modo que pares com menor impacto na coancestraldade tenham maior probabilidade de seleção

DEFINIÇÕES EM ACS

- **Cada formiga percorre o grafo selecionado, para cada fêmea, um macho válido é escolhido por meio de uma regra probabilística que combina o feromônio e a heurística, sendo ela: $\tau_{(i,j)}^{\alpha}\eta_{(i,j)}^{\beta}$**

- **Resumindo:**
  M = conjunto de touros F = conjunto de vacas
  (i, j) = descendente dos dois animais onde i $\in$ M e j $\in$ F
  $\tau_{(i,j)}$ = nível de feromônio (α)
  $\eta_{(i,j)}$ = heurística de visibilidade (β)
  $\tau_{(i,j)}^{\alpha}\eta_{(i,j)}^{\beta}$ = probabilidade de escolha (regra local)

IMPLEMENTAÇÃO E ESTRATÉGIAS

- Linguagem: Python (Bibliotecas: Pandas, NumPy)
- Pré-processamento:
  - Mapeamento de IDs para índices matriciais (0 a N).
  - Otimização: Criação de HashMap (Dicionário) para acesso à coancestralidade em tempo O(1), reduzindo o custo computacional de $O(N^2)$.
- Tratamento da Heurística:
  - Índices genéticos negativos não são descartados ou modificados;
- Menu interativo:
  - Permite escolher o arquivo a ser importado;
  - Permite definir o limite de uso dos machos;
  - Imprime os resultados obtidos como valor máximo de coancestralidade, valor médio e máximo de índice genético, tempo de execução, etc.

DEFINIÇÕES MATRIZ E NORMALIZAÇÃO

Matriz de Heurística:

- Representa a atratividade genética (ganho esperado) de acasalar a Fêmea j com o Macho i
- Fórmula: Média dos índices genéticos dos pais: (Índice(i) + Índice(j))/2

O Problema dos Valores Negativos:

- Índices genéticos reais podem ser negativos (abaixo da média da raça);
- A fórmula de probabilidade da ACS exige valores positivos;
- Se n<=0, a equação matemática quebra ou zera a probabilidade incorretamente;
- Aplicamos uma transformação linear para garantir que o menor valor possível seja positivo, onde n_final=n_original + min(n) + 1

$$P_{(i,j)} = \frac{[t_{(i,j)}] * [n_{(i,j)}]}{\sum([t]*[n])}$$

EXEMPLO

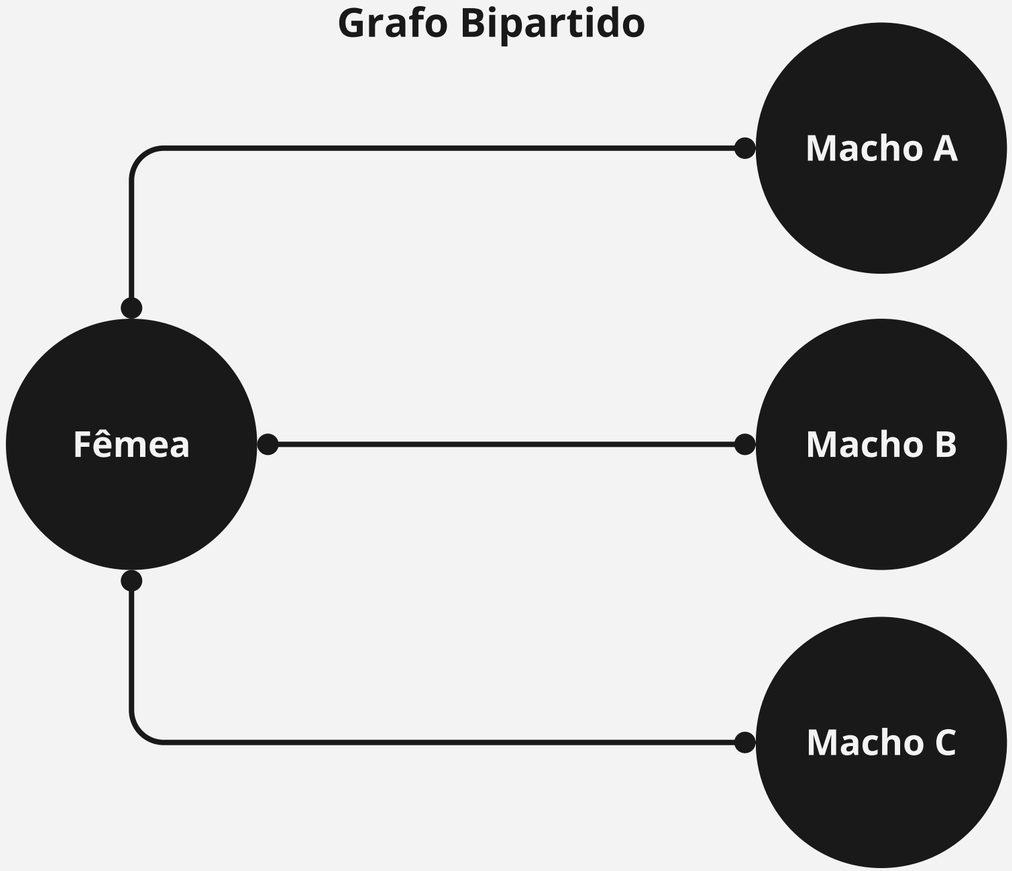
A Fêmea 01 precisa escolher um parceiro. Existem 3 Machos disponíveis.
(Assumindo alpha e beta como 1)

- Macho A: 2.0 x 10 = 20
- Macho B: 8.0 x 5 = 40
- Macho C: Excelente, porém atingiu o número máximo de usos;

- Chance do Macho A: 20 / 60 = 33.3%
- Chance do Macho B: 40 / 60 = 66.6%
- Chance do Macho C: 0%

Apesar do Macho A ter genética melhor, o algoritmo favoreceu o Macho B (66%) porque a colônia 'aprendeu' (feromônio) que ele gera resultados melhores no conjunto global

MODELO



A Fêmea 01 precisa escolher um parceiro.
Existem 3 Machos disponíveis
A grossura da linha ou a probabilidade de escolha depende de dois fatores:

1. O Feromônio (tau): O quanto aquele touro foi bom para o grupo no passado
2. A Heurística: O ganho genético imediato daquele casal. Tudo isso respeitando a restrição de que um nó do lado dos Machos só pode receber um número limitado de conexões (ex: 12, 18 ou 20).

ETAPA 1

CARREGANDO ARQUIVOS

```
print("\nETAPA 1: CARREGANDO ARQUIVOS\n")

url_parentesco_produtos = "https://raw.githubusercontent.com/Thalyszor/antcolony
url_pais_indices = "https://raw.githubusercontent.com/Thalyszor/antcolonyoptmiza
url_pais_indices_modificados = "https://raw.githubusercontent.com/Thalyszor/antc

df_coancestralidade = pd.read_csv(url_parentesco_produtos, sep=',')
df_coancestralidade.columns = df_coancestralidade.columns.str.strip().str.lower(
if len(df_coancestralidade.columns) == 3:
    df_coancestralidade.columns = ['animal1', 'animal2', 'coef']
print("Arquivo 'parentesco_produtos.csv' carregado e padronizado.")

df_indice = pd.read_csv(url_pais_indices_modificados, sep=',')
df_indice.columns = [col.strip().lower() for col in df_indice.columns]
if 'animal' in df_indice.columns[0] and 'indice' in df_indice.columns[1]:
    df_indice.columns = ['animal_id', 'indice']
print("Arquivo 'pais_indice.csv' carregado e padronizado.")
```

- A biblioteca pandas transforma cada um dos arquivos csv em um dataframe;
- Limpa os nomes das colunas (tira espaços, deixa minúsculas);
- Padroniza os nomes para garantir que animal_id seja sempre animal_id
- 3 opções: Arquivo original, modificado ou escolher um arquivo do computador

ETAPA 2

MAPEANDO MACHOS (M) E FÊMEAS (F)

```
print("\nETAPA 2: MAPEANDO MACHOS (M) E FÊMEAS (F)\n")

macho_ids_set = set()
femea_ids_set = set()

def mapear_animal(animal_str):
    try:
        macho_str, femea_str = str(animal_str).split('_')
        macho_ids_set.add(int(macho_str))
        femea_ids_set.add(int(femea_str))
    except (ValueError, TypeError):
        pass

df_coancestralidade['animal1'].apply(mapear_animal)
df_coancestralidade['animal2'].apply(mapear_animal)

lista_machos = sorted(list(macho_ids_set))
lista_femeas = sorted(list(femea_ids_set))
num_femeas = len(lista_femeas)
num_machos = len(lista_machos)

print(f"Total de Fêmeas únicas |F|: {num_femeas}")
print(f"Total de Machos únicos |M|: {num_machos}")
```

- Separa os IDs que vieram grudados no arquivo de parentesco (transforma o texto "Macho_Fêmea" em dois números separados)
- Remove duplicatas para criar uma lista de animais únicos (usando set), já que o mesmo animal aparece repetido em vários cruzamentos
- Ordena e conta os animais para definir o tamanho exato da matriz que o algoritmo vai usar (quantas linhas e quantas colunas)

ETAPA 3

PREPARANDO DATAFRAMES

```
print("\nETAPA 3: PREPARANDO DATAFRAMES\n")

df_indice['animal_id'] = pd.to_numeric(df_indice['animal_id'], errors='coerce')
df_indice.dropna(subset=['animal_id'], inplace=True)
df_indice['animal_id'] = df_indice['animal_id'].astype(int)

df_machos = pd.DataFrame(lista_machos, columns=['animal_id'])
df_femeas = pd.DataFrame(lista_femeas, columns=['animal_id'])

df_machos = pd.merge(df_machos, df_indice, on='animal_id', how='left')
df_femeas = pd.merge(df_femeas, df_indice, on='animal_id', how='left')

df_femeas['indice'] = df_femeas['indice'].fillna(0)
df_machos['indice'] = df_machos['indice'].fillna(0)
```

- Valida os IDs, garantindo que sejam números inteiros e removendo qualquer erro de formatação;
- Transforma as listas de machos e fêmeas em DataFrames;
- Preenche o "valor genético" (índice) de cada animal e assume valor 0 para qualquer animal que não tenha dados cadastrados

ETAPA 4

CONSTRUINDO MATRIZ DE HEURÍSTICA

```
print("\nETAPA 4: CONSTRUINDO MATRIZ DE HEURÍSTICA\n")

indices_femeas = df_femeas['indice'].values
indices_machos = df_machos['indice'].values

soma_indices = indices_femeas.reshape(-1, 1) + indices_machos.reshape(1, -1)
matriz_indices_reais = soma_indices / 2

min_val = np.min(matriz_indices_reais)
if min_val <= 0:
    print(f"Detectados valores negativos/zero ({min_val}). Normalizando matriz
    matriz_heuristica_eta = matriz_indices_reais + abs(min_val) + 1.0
else:
    matriz_heuristica_eta = matriz_indices_reais

print(f"Matriz de Heurística pronta. Shape: {matriz_heuristica_eta.shape}")
```

- Calcula o valor genético dos cruzamentos possíveis (índ. Pai + índ. Mãe / 2)
- Normaliza a matriz;
- Gera a Matriz Heurística, garantindo que todos os números sejam positivos para utilizar a fórmula de probabilidade do ACO.

ETAPA 5

INDEXANDO COANCESTRALIDADE (HASHMAP)

```
print("\nETAPA 4.5: INDEXANDO COANCESTRALIDADE (HASHMAP)\n")

coancestralidade_dict = {}
termos_a = df_coancestralidade['animal1'].values
termos_b = df_coancestralidade['animal2'].values
coefs = df_coancestralidade['coef'].values

def parse_key(termo):
    m, f = termo.split('_')
    return (int(m), int(f))

print("Criando dicionário rápido...")
for i in range(len(coefs)):
    key_a = parse_key(termos_a[i])
    key_b = parse_key(termos_b[i])
    val = coefs[i]
    coancestralidade_dict[(key_a, key_b)] = val
    coancestralidade_dict[(key_b, key_a)] = val

print(f"Dicionário criado com {len(coancestralidade_dict)} entradas.")
```

- Converte a tabela lenta do Pandas em um Hashmap, permitindo acesso instantâneo aos dados (O(1))
- Duplica as chaves de busca para funcionar nos dois sentidos (A com B, e B com A)
- Otimiza a performance, evitando que o algoritmo trave ao ter que consultar milhares de vezes a mesma tabela durante o cálculo de custo

ETAPA 6

```
for ant in range(NUM_FORMIGAS):
    caminho_formiga = []
    uso_machos = {m_idx: 0 for m_idx in range(num_machos)}
    valid_path = True

    for i_f in range(num_femeas):
        probabilidades = []
        candidatos_validos = []

        for i_m in range(num_machos):
            if uso_machos[i_m] >= MAX_USO_MACHO:
                prob = 0.0
            else:
                t = tau[i_f, i_m] ** ALPHA
                n = matriz_heuristica_eta[i_f, i_m] ** BETA
                prob = t * n
                candidatos_validos.append(i_m)
            probabilidades.append(prob)

        denominador = sum(probabilidades)

        if denominador > 0:
            probabilidades = [p / denominador for p in probabilidades]
            escolha_macho_idx   np.random.choice(range(num_machos), p=probabilidades)
```

- Em cada iteração, várias formigas constroem casais usando uma roleta probabilística, garantindo que machos que já atingiram o limite de uso sejam ignorados;
- A escolha favorece pares com maior índice genético;
- A roleta (np.random.choice) permite que, ocasionalmente, caminhos menos óbvios sejam testados, evitando ótimos locais

```
pares_reais = []
for i_f, i_m in caminho_formiga:
    id_f = lista_femeas[i_f]
    id_m = lista_machos[i_m]
    pares_reais.append((id_m, id_f))
    sum_indice_genetico += matriz_indices_reais[i_f, i_m]

n_pares = len(pares_reais)
for i in range(n_pares):
    for j in range(i + 1, n_pares):
        par_a = pares_reais[i]
        par_b = pares_reais[j]
        cost_coancestralidade += coancestralidade_dict.get((par_a, par_b), 0.0)

solutions_in_iter.append({
    'caminho': caminho_formiga,
    'custo': cost_coancestralidade,
    'indice_total': sum_indice_genetico
})

if cost_coancestralidade < best_cost:
    best_cost = cost_coancestralidade
    best_solution = caminho_formiga
    best_index_sum = sum_indice_genetico
elif cost_coancestralidade == best_cost:
    if sum_indice_genetico > best_index_sum:
        best_solution = caminho_formiga
        best_index_sum = sum_indice_genetico

*= (1   EVAPORATION)

sol in solutions_in_iter:
custo = sol['custo']
if custo <= 0.0001: custo = 0.0001

delta = Q / custo

for i_f, i_m in sol['caminho']:
    tau[i_f, i_m] += delta
```

ETAPA 6

- O custo é a soma das coancestralidades de todos os pares da solução.
- Uso de dicionário para acesso imediato.
- Mantém a melhor configuração encontrada até o momento (Elitismo).
- Evaporação: O ambiente "esquece" caminhos antigos.
- Reforço: Soluções de alta qualidade (baixo parentesco) recebem mais feromônio, atraindo futuras formigas.

CUSTO DOMINANTE

COANCESTRALIDADE O((i_j)²)

```
for i in range (n_pares):
    for j in range(i + 1, n_pares):
        dict += dict.get((par_a, par_b), 0)
```

CONSTRUÇÃO DE CAMINHO O(i*j)

```
for i_f in range (num_femeas):
  prob = []
  cand_validos = []
  for i_m in range (num_machos):
    if uso_machos[i_m] >= max_uso:
       prob = 0
  else:
     t = tau[i_f,i_m] * ALPHA
     n = matriz[i_f][i_m] * BETA
     prob = t * n
     cand_validos.append(i_m)
   prob.append(prob)
```

I_j representa o número de pares

CUSTO DOMINANTE

- O custo de construir o caminho das formiga é O(i*j), onde, dependendo do contexto, pode vir a ser O((i_j)²) dependendo do número de formigas;
- Dessa forma, a complexidade final do algoritmo também deve incluir I iterações e A formigas, além dos i_j pares:

$$O(I * A *(n^2 + i * j))$$

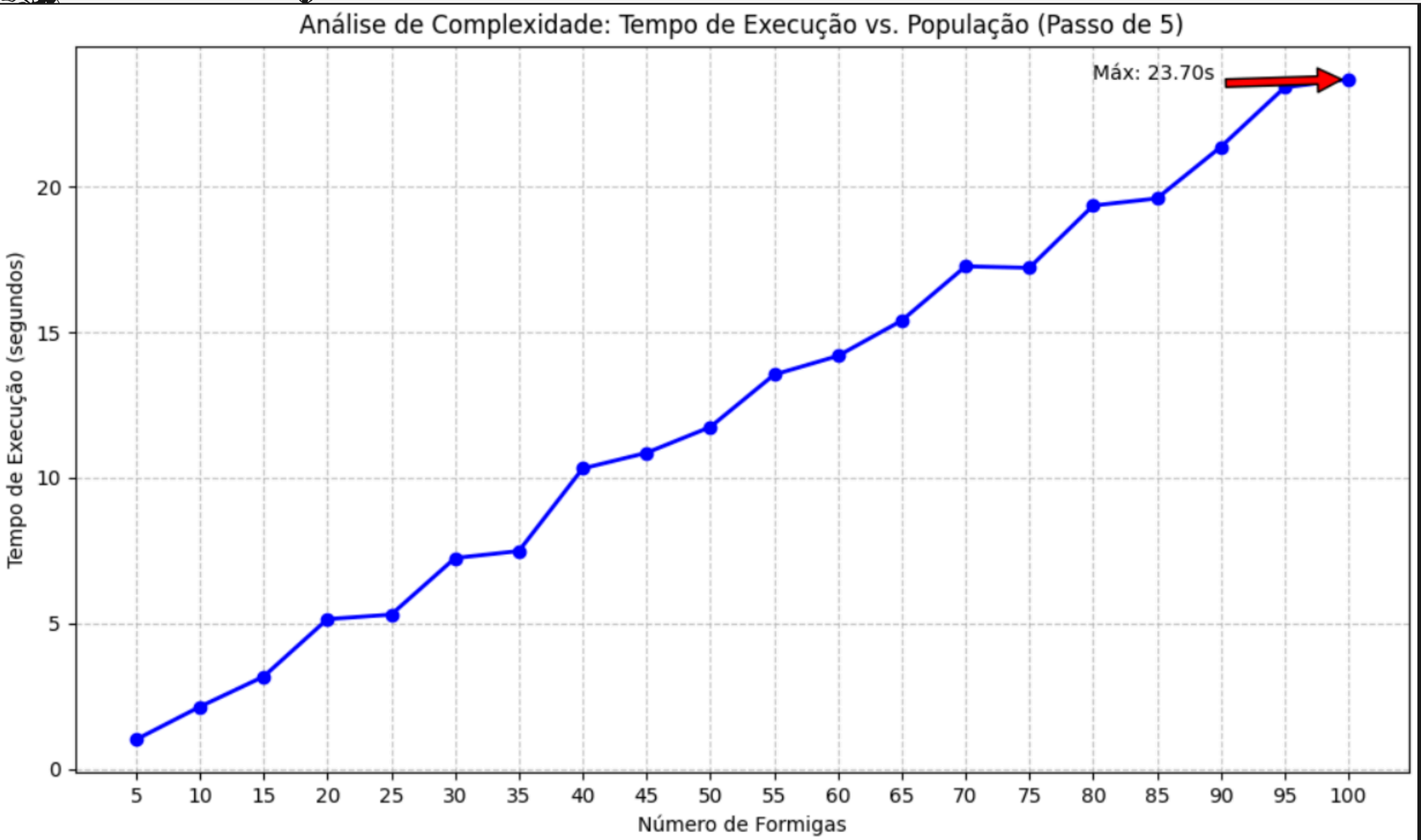
n²: É o custo de calcular a Coancestralidade (avaliar se a solução é boa). Onde n é o número de pares (fêmeas).

i*j: É o custo de construir o caminho (visitar cada fêmea i e olhar os machos j).

- Que pode ser simplificado em, considerando que o termo dominante do parêntese é n²:

$$O(I * A * n^2)$$

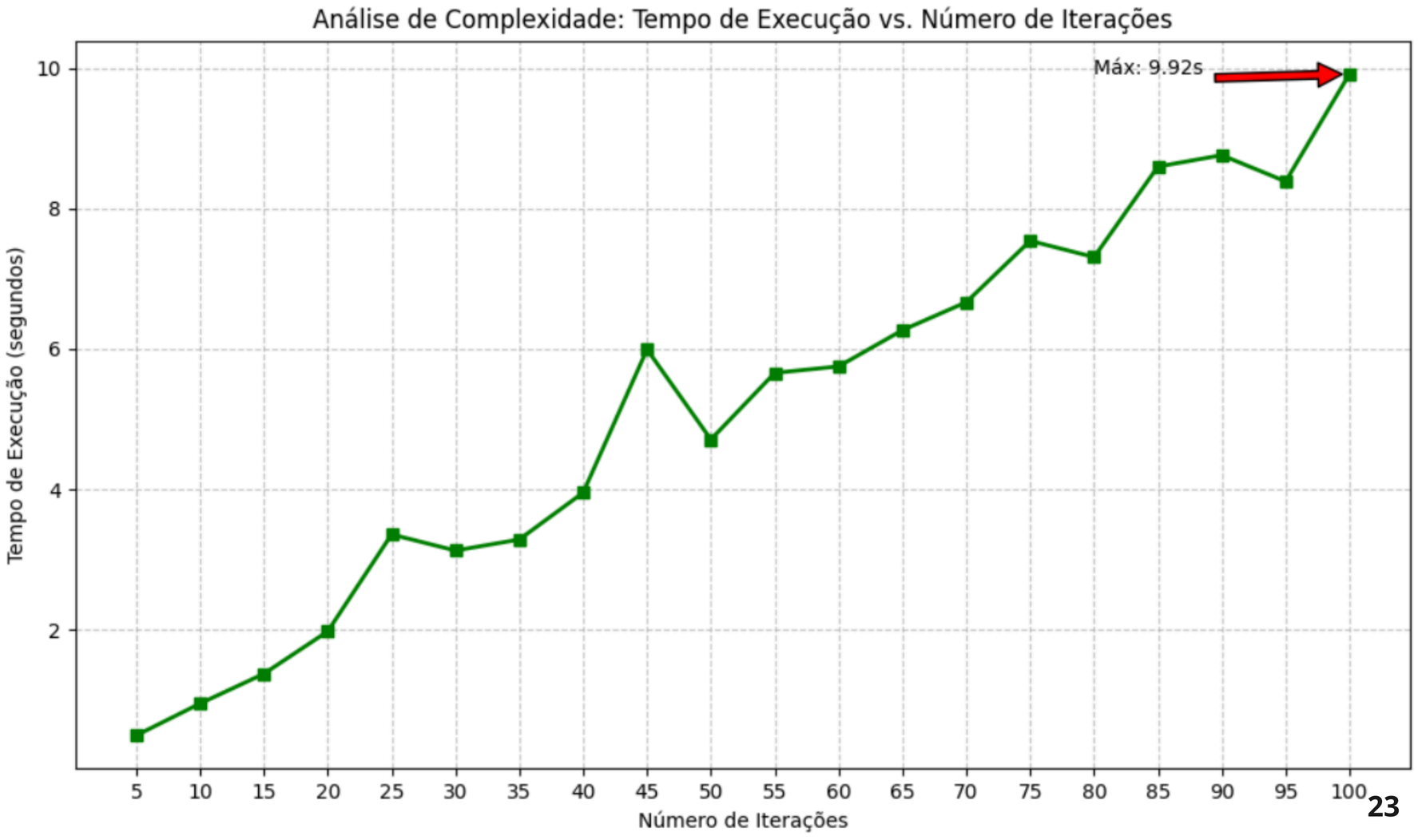
ANÁLISE DE COMPLEXIDADE EXPERIMENTAL



- Cada formiga constrói uma solução completa em cada iteração;
- Aumentar o número de formigas aumenta diretamente o custo computacional.

ANÁLISE DE COMPLEXIDADE EXPERIMENTAL

- Cada iteração envolve construção de todas as soluções e a atualização global dos feromônios;
- Portanto, o custo cresce proporcionalmente ao número de ciclos da colônia.



TESTES

PARÂMETROS

- População: 60 Fêmeas | 5 Machos
- Configuração do ACO:
  - Número de formigas artificiais: 20
  - Número de iterações: 50
  - Importância do Feromônio α: 1.0
  - Importância da Heurística β: 3.0 (Prioridade alta para o Índice Genético)
  - Taxa de Evaporação: 0.5
- Cenários Testados:
  - Considerando apenas 5 machos para 60 fêmeas, o menor valor para testes é 12, onde foram testados limites até 20 usos.

Números baixos fariam a colonia não aprender a tempo, enquanto números altos demais aumentariam o custo computacional sem ganho significativo

RESULTADOS OBTIDOS

- Média dos resultados obtidos após 30 execuções do algoritmo para cada um dos limites;
- Os valores mostram um comportamento de troca proporcional, onde liberar mais acasalamentos por macho favorece soluções geneticamente superiores ao custo de maior concentração genética.

LIMITE	COANCESTRALIDADE	SOMA ÍNDICES GENÉTICOS
12	83,32	77.50
13	83,85	82,50
14	84,93	87,50
15	85,09	92,50
16	86,74	97,50
17	88,44	102,50
18	91.43	107.50
19	93,89	112,50
20	96.21	117.50

LIMITE IDEAL DADO OS TESTES

- A escolha do limite ideal depende do balanço entre conservação e ganho genético dentro do sistema de seleção. Para permitir a comparação equilibrada entre os esses parâmetros, normalizamos os dados da solução em um intervalo de [0, 1], seguindo as fórmulas abaixo.

COANCESTRALIDADE

$$C_L' = \frac{C_L - \min(C)}{\max(C) - \min(C)}$$

SOMA DE ÍNDICES

$$S_L' = \frac{S_L - \min(S)}{\max(S) - \min(S)}$$

UTILIDADE

$$U(L) = w_s * S_L' - w_c * C_L'$$

LIMITE	C_L'	S_L'	U(L)
12	0,000	0,000	0,000
13	0,041	0,125	0,084
14	0,125	0,250	0,125
15	0,137	0,375	0,238
16	0,266	0,500	0,234
17	0,397	0,625	0,228
18	0,631	0,750	0,119
19	0,822	0,875	0,053
20	1,000	1,000	0,000

CONSIDERAÇÕES FINAIS

- A análise evidencia que nenhum limite de uso é simultaneamente ótimo para ambos os critérios, pela função de utilidade adotada observa-se que 15 limitações do uso dos machos ofereceu o cenário mais vantajoso dentre os avaliados;

- O método Ant Colony Optmization mostrou-se eficaz para resolver o problema de acasalamento, equilibrando ganho genético e controle de coancestralidade;

- O trabalho pode ser expandido para incluir diferentes fatores como: funções de utilidade adaptativas, inclusão de custos econômicos, análise multigeracional e comparação com outras meta-heurísticas.

REFERÊNCIAS

- DORIGO, Marco; SOCHA, Krzysztof. An introduction to ant colony optimization. In: Handbook of approximation algorithms and metaheuristics. Chapman and Hall/CRC, 2018. p. 395-408.

- DORIGO, Marco; GAMBARDELLA, Luca Maria. Ant colony system: a cooperative learning approach to the traveling salesman problem. IEEE Transactions on evolutionary computation, v. 1, n. 1, p. 53-66, 1997.

- MCKINNEY, Wes. Data structures for statistical computing in python. In: Proceedings of the 9th Python in Science Conference. 2010. v. 445, p. 51-56.

- MANIEZZO, Vittorio; GAMBARDELLA, Luca Maria; DE LUIGI, Fabio. Ant colony optimization. In: New optimization techniques in engineering. Berlin, Heidelberg: Springer Berlin Heidelberg, 2004. p. 101-121.





DÚVIDAS?